

Tâche 3

Qualité du logiciel et métriques
IFT 3913

Par

Heyun Liu (20269538)

Et

Kevin Guo (20275841)

Présenté à

Benoit Baudry

21 novembre 2025

Universite de Montreal

1 Objectif

L'objectif de cette modification du workflow GitHub Actions de GraphHopper est de garantir que la qualité des tests unitaires ne se dégrade pas au fil des commits. Le processus de build doit échouer automatiquement si le *mutation score* (score PITest) diminue par rapport au baseline.

2 Approche

L'approche retenue consiste à intégrer le mutation testing PITest dans un job dédié du pipeline CI/CD :

- exécute PITest sur un ensemble de classes selectionnées ;
- calcule automatiquement le score de mutation à partir du rapport XML ;
- compare ce score à un baseline dans le dépôt Git ;
- fait échouer le workflow en cas de régression du score.

3 Choix de conception

3.1 Outil : PITest

Nous utilisons PITest (version 1.9.0) avec le plugin JUnit 5.

3.2 Scope limité à deux classes

Afin de limiter le temps d'exécution en CI, le mutation testing ne cible que deux classes du module *core* :

- `com.graphhopper.util.DistanceCalcEuclidean` ;
- `com.graphhopper.util.shapes.Circle`.

Ce scope restreint permet d'obtenir un retour en 1–2 minutes, tout en démontrant le fonctionnement du système.

3.3 Architecture du workflow (build.yml)

Un job séparé `mutation-testing` est ajouté au workflow GitHub Actions :

- il est indépendant du job `build` exécutant les tests unitaires classiques ;
- il est configuré avec `if: always()` pour s'exécuter même si d'autres jobs échouent ;
- il est responsable de l'exécution de PITest, de l'extraction du score et de la comparaison au baseline.

3.4 Baseline stocké dans Git

Le baseline du score de mutation est stocké dans un fichier texte commité :

`.github/mutation-baseline.txt`

Ce choix offre :

- une traçabilité avec l'historique Git ;
- une simplicité de changement ;
- une visibilité facile dans les pull requests.

La règle de mise à jour :

- mise à jour uniquement sur la branche `main` ;
- la valeur du baseline ne peut jamais être diminuée automatiquement.

4 Implémentation

4.1 Configuration PITest

Dans `core/pom.xml`, le plugin PITest est configuré pour :

- cibler uniquement les classes `DistanceCalcEuclidean` et `Circle` ainsi que leurs tests associés ;
- générer un rapport XML (`mutations.xml`) utilisé pour le calcul du score ;
- utiliser le plugin JUnit 5.

4.2 Étapes principales du workflow

Le job `mutation-testing` exécute les étapes suivantes :

1. **Build sans tests classiques** : `mvn -B clean install -DskipTests`
2. **Compilation des tests du module core** : `mvn -B test-compile`
3. **Exécution de PITest** : `mvn -B org.pitest:pitest-maven:mutationCoverage`
4. **Extraction du score de mutation** à partir de `mutations.xml` via un script Python :

```
import xml.etree.ElementTree as ET
tree = ET.parse('mutations.xml')
root = tree.getroot()
mutations = root.findall('mutation')
total = len(mutations)
killed = sum(1 for m in mutations if m.get('status') == 'KILLED')
score = (killed * 100.0 / total) if total > 0 else 0
print(f"{score:.2f}")
```

5. **Comparaison au baseline** :
 - si `score < baseline` alors le job échoue (`exit 1`) ;
 - sinon le job réussit, et sur `main` le baseline peut être mis à jour.

4.3 Optimisations du Workflow CI/CD

Plusieurs optimisations ont été ajoutées au workflow GitHub Actions afin de réduire le temps d'exécution du pipeline tout en préservant la fiabilité du processus de validation.

1. Exécution ciblée du module core

GraphHopper est un projet multi-modules (`core`, `web`, `navigation`, etc.). Tester tous les modules à chaque exécution du pipeline serait vraiment long

Comme les tests ajoutés (dont `CircleTest`) se trouvent dans le module `core`, le workflow CI exécute uniquement les tests de ce module :

```
- name: Build and test only core module
  run: mvn -B clean test -pl core -am
```

L'option `-pl core` limite la construction au module ciblé, tandis que `-am` ("also make") compile automatiquement les dépendances nécessaires. Cette optimisation réduit le temps d'exécution

2. Gestion du baseline de mutation testing

Le workflow compare automatiquement le score courant de mutation testing au baseline stocké dans `.github/mutation-baseline.txt` :

```
- name: Compare mutation scores
  run: |
    BASELINE="${{ steps.baseline_score.outputs.baseline_score }}"
    CURRENT="${{ steps.current_score.outputs.current_score }}"

    echo "Mutation Testing Analysis"
    echo "Baseline score: $BASELINE%"
    echo "Current score: $CURRENT%"

    if (( $(awk "BEGIN {print ($CURRENT < $BASELINE)}") )); then
      echo "FAILURE: Mutation score has decreased"
      exit 1
    fi
```

Le workflow échoue en cas de régression, garantissant qu'aucun commit ne diminue la qualité des tests.

Nous avons volontairement choisi de **ne pas automatiser** la mise à jour du baseline, même si GitHub Actions le permettrait via un script comme celui-ci :

```
- name: Update baseline if on main branch
  if: github.ref == 'refs/heads/main'
  run: |
    echo "${{ steps.current_score.outputs.current_score }}" > .github/mutation-baseline.
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add .github/mutation-baseline.txt
    git commit -m "Update mutation baseline"
    git push
```

Cette automatisation n'a pas été retenue pour trois raisons :

- **Permissions GitHub** : un *Personal Access Token* ou des permissions élevées seraient nécessaires, ce qui introduit un risque de sécurité.
- **Validation humaine** : une hausse du score doit être vérifiée manuellement, pour éviter qu'une fausse amélioration invalide la référence historique.
- **Stabilité du baseline** : une mise à jour automatique pourrait faire grimper progressivement le baseline jusqu'à un niveau irréaliste (par exemple 100 %).

La solution retenue est donc :

- détection automatique des régressions (*fail* si le score baisse),
- affichage du score amélioré dans les logs,
- mise à jour manuelle et contrôlée de `mutation-baseline.txt`.

Cette approche offre un équilibre optimal entre automatisation et contrôle humain.

3. Limitation du mutation testing à deux classes

Afin de réduire le temps de calcul du mutation testing, PITest est configuré pour cibler uniquement deux classes du module `core` :

- `com.graphhopper.util.DistanceCalcEuclidean`,
- `com.graphhopper.util.shapes.Circle`.

Pour tester tout le module `core` avec PITest, il suffit de retirer les balises `targetClasses` et `targetTests` dans la configuration du plugin PITest dans `pom.xml`. Sans ces restrictions, PITest testera automatiquement toutes les classes du module. Par contre, ça prendra un temps fou lieu de 1-2 minutes pour tester le score du mutation de tous ses classes.

5 Validation

5.1 Scénario de régression volontaire

Un baseline initial de **57 %** a été enregistré dans `.github/mutation-baseline.txt`. Pour valider le mécanisme de détection de régression, une modification affaiblissant les tests a été introduite (on a fait la suppression d'assertions dans un test lié à la classe `Circle`).

L'exécution du workflow a alors produit un score courant de **55.28 %**. La différence de **-1.72 point de pourcentage** a été détectée et le job `mutation-testing` a échoué, ce qui prouve que la régression est correctement bloquée avant le merge.

```
@Test
public void testContains() {
    Circle c = new Circle(10, 10, 120000);
    assertTrue(c.contains(new BBox(9, 11, 10, 10.1)));
    // assertFalse(c.contains(new BBox(9, 11, 8, 9)));
    // assertFalse(c.contains(new BBox(9, 12, 10, 10.1)));
}

@Test
public void testContainsCircle() {
    Circle c = new Circle(10, 10, 120000);
```

```

    assertTrue(c.contains(new Circle(9.9, 10.2, 90000)));
    // assertFalse(c.contains(new Circle(10, 10.4, 90000)));
}

```

5.2 Scénario d'amélioration

Les tests ont ensuite été corrigés et renforcés. Un nouveau run du workflow a donné un score de mutation de **57.14 %**. La différence par rapport au baseline est de **+0.14 point de pourcentage**. Le job a alors réussi, validant que le système accepte les améliorations tout en conservant le baseline comme référence. Note : Les résultats sont visibles directement dans les exécutions des workflows #50 et #51 dans notre github.

5.3 Captures d'écran du workflow GitHub Actions

Les Figures 1 et 2 illustrent ces deux exécutions.

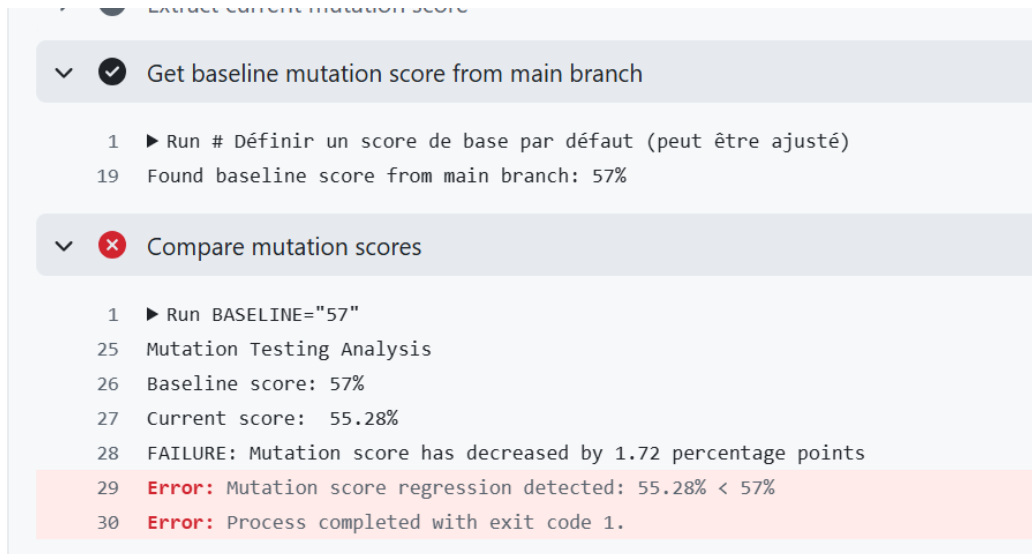


FIGURE 1 – Échec du job `mutation-testing` : baseline 57 %, score courant 55.28 %

```

  ✓ Get baseline mutation score from main branch

  1 ▶ Run # Définir un score de base par défaut (peut être ajusté)
  19 Found baseline score from main branch: 57%

  ✓ Compare mutation scores

  1 ▶ Run BASELINE="57"
  25 Mutation Testing Analysis
  26 Baseline score: 57%
  27 Current score: 57.14%
  28 SUCCESS: Mutation score has improved by 0.14 percentage points

  ✓ Update baseline if on main branch

  1 ▶ Run echo "Current mutation score: 57.14%"
  8 Current mutation score: 57.14%
  9 To update the baseline, manually edit .github/mutation-baseline.txt and commit the change

```

FIGURE 2 – Succès du job `mutation-testing` : baseline 57%, score courant 57.14% (amélioration de 0.04 point de pourcentage).

6 Ajout de tests Mockito pour la classe `Circle`

Dans le cadre de l'amélioration de la couverture de tests et de l'isolation des composants, des tests unitaires supplémentaires utilisant Mockito ont été ajoutés pour la classe `Circle` du module `core` de `GraphHopper`.

6.1 Résumé des modifications

Quatre tests unitaires basés sur Mockito ont été intégrés dans le fichier `CircleTest.java`. Ces tests permettent de vérifier la logique interne de `Circle` sans dépendre des calculs géographiques réels ou de structures complexes provenant du projet.

6.2 Classes mockées

Deux classes ont été simulées via Mockito :

- **`DistanceCalc`** (interface) – mockée dans trois tests. Elle constitue la dépendance principale utilisée par `Circle` pour effectuer des calculs géodésiques. Le mock permet d'isoler la logique métier et de définir des valeurs déterministes pour les distances.
- **`PointList`** (classe concrète) – mockée dans un test. Cette classe est utilisée pour représenter des polygones géographiques. Le mock permet de contrôler précisément le nombre de points et leurs coordonnées pour tester la méthode `intersects()`.

6.3 Tests créés

Les quatre tests ajoutés sont les suivants :

- `testContainsWithMockedDistanceCalc` : vérifie qu'un point se trouvant dans le cercle est correctement détecté lorsque la distance mockée est inférieure au rayon.
- `testIntersectsWithMockedDistanceCalc` : teste l'intersection d'un segment géographique avec le cercle en simulant les distances au segment.
- `testContainsPointOutsideWithMock` : vérifie qu'un point situé hors du cercle est correctement rejeté.
- `testIntersectsWithMockedPointList` : teste la logique d'intersection à l'aide d'une `PointList` entièrement simulée.

6.4 Dépendances ajoutées

Les dépendances suivantes ont été ajoutées au `pom.xml` du module `core` :

- `mockito-core` (version 5.8.0)
- `mockito-junit-jupiter` (version 5.8.0)

Ces dépendances permettent l'utilisation de Mockito ainsi que son intégration avec JUnit 5.

6.5 Documentation

Chaque test ajouté est accompagné d'une documentation interne détaillée dans `CircleTest.java`. Les commentaires expliquent :

- le scénario testé,
- les valeurs mockées,
- les raisons du mock,
- les comportements attendus.

Cette documentation permet de maintenir une compréhension claire du rôle de chaque test lors des futures évolutions de la classe `Circle`.

6.6 Exécution des tests du module `core`

Les tests ajoutés pour la classe `Circle` appartiennent au module `core` du projet `GraphHopper`. Comme il s'agit d'un projet multi-modules, il n'est pas recommandé d'exécuter un test d'un module donné depuis la racine du projet : Maven tenterait alors de lancer ce test dans tous les modules, y compris ceux dont la responsabilité est différente (comme `graphhopper-web-api`), et qui ne disposent pas de cette classe de test.

Pour respecter la structure modulaire du projet et exécuter uniquement les tests pertinents, il est nécessaire de cibler explicitement le module `core`. La commande suivante permet de lancer la classe `CircleTest` :

```
mvn test -pl core -Dtest=com.graphhopper.util.shapes.CircleTest
```

Cette approche garantit que seuls les tests du module concerné sont exécutés, et évite l'exécution inutile ou incorrecte des tests dans les autres modules du projet.

7 Easter Egg Rickroll dans GitHub Actions

Un petit *easter egg* humoristique a été ajouté au workflow GitHub Actions : lorsqu'un test échoue, un message affichant le lien vers la vidéo *Never Gonna Give You Up* apparaît dans les logs.

L'étape suivante a été ajoutée dans `build.yml` :

```
- name: Rickroll on test failure
  if: failure()
  run: |
    echo "Tests failed! Click on this link to fix the problem"
    echo "https://www.youtube.com/watch?v=dQw4w9WgXcQ"
```

Validation

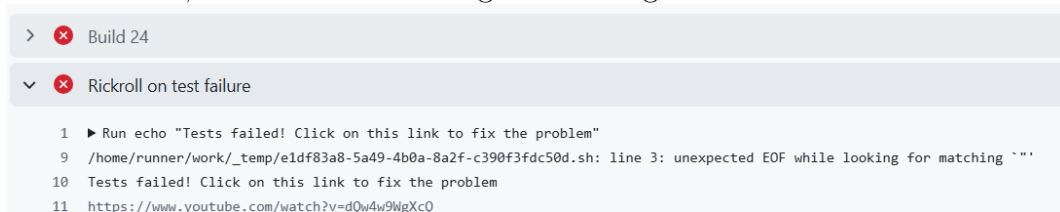
Pour tester l'easter egg, un échec volontaire a été introduit dans `CircleTest` en remplaçant :

```
assertTrue(c.contains(new Circle(9.9, 10.2, 90000)));
```

par :

```
assertFalse(c.contains(new Circle(9.9, 10.2, 90000)));
```

Le test échoue alors, déclenchant l'affichage du message :



```
> ✖ Build 24
✓ ✖ Rickroll on test failure
1 ▶ Run echo "Tests failed! Click on this link to fix the problem"
9 /home/runner/work/_temp/e1df83a8-5a49-4b0a-8a2f-c390f3fdc50d.sh: line 3: unexpected EOF while looking for matching `"'
10 Tests failed! Click on this link to fix the problem
11 https://www.youtube.com/watch?v=dQw4w9WgXcQ
```

Cet easter egg n'affecte pas le pipeline, mais rend les échecs de tests plus amusants.